实验目的：

熟悉并掌握 Socket 网络编程的过程与技术；深入理解 HTTP 协议，掌握 HTTP 代理服务器的基本工作原理；掌握 HTTP 代理服务器设计与编程实现的基本技能。

实验内容：

(1) 设计并实现一个基本 HTTP 代理服务器。要求在指定端口（例如8080）接收来自客户的 HTTP 请求并且根据其中的 URL 地址访问该地址所指向的 HTTP 服务器（原服务器），接收 HTTP 服务器的响应报文，并将响应报文转发给对应的客户进行浏览。

(2) 设计并实现一个支持 Cache 功能的 HTTP 代理服务器。要求能缓存原服务器响应的对象，并能够通过修改请求报文（添加 if-modified-since 头行），向原服务器确认缓存对象是否是最新版本。

(3) 扩展 HTTP 代理服务器，支持如下功能：

a) 网站过滤：允许/不允许访问某些网站；

b) 用户过滤：支持/不支持某些用户访问外部网站；

c) 网站引导：将用户对某个网站的访问引导至一个模拟网站（钓鱼）。

实验过程：

一．Socket 编程的客户端和服务器端主要步骤

客户端：

1. 确认目的服务器IP地址和端口号，并创建相应的套接字，连接服务器
2. 发送请求报文
3. 接受返回报文
4. 关闭连接

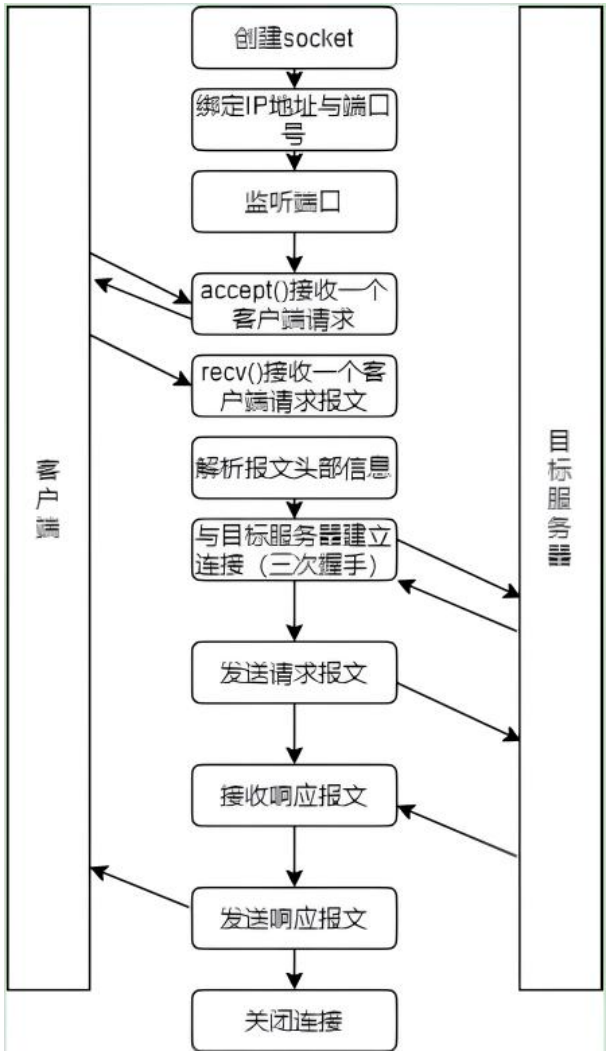
服务器端：

1. 创建套接字，绑定IP地址和端口号，监听端口
2. 从连接队列中取出一个连接请求，三次握手创建连接
3. 接收请求报文
4. 发送响应报文
5. 关闭当前连接，继续监听端口

二．代理服务器的基本原理

代理服务器，俗称“翻墙软件”，允许一个网络终端（一般为客户端）通过这个服务与另一个网络终端（一般为服务器）进行非直接的连接代理服务器在指定端口（例如8080）监听浏览器的访问请求（需要在客户端浏览器进行相应的设置），接收到浏览器对远程网站的浏览请求时，代理服务器开始在代理服务器的缓存中检索URL对应的对象（网页、图像等对象），找到对象文件后，提取该对象文件的最新被修改时间；代理服务器程序在客户的请求报文首部插入<If-Modified-Since: 对象文件的最新被修改时间>，并向原Web 服务器转发修改后的请求报文。如果代理服务器没有该对象的缓存，则会直接向原服务器转发请求报文，并将原服务器返回的响应直接转发给客户端，同时将对象缓存到代理服务器中。代理服务器程序会根据缓存的时间、大小和提取记录等对缓存进行清理。

三．HTTP代理服务器的程序流程图



首先需要设计基本的处理客户HTTP请求的函数，为此我们设计了handleClient函数，在建立连接后，接收浏览器发来的报文。

为了分析浏览器的请求，我们编写新的函数parseHostFromRequest()，由于HTTP请求报文类似"GET /index.html HTTP/1.1\r\nHost: www.example.com\r\n..."(否则不是合法请求)，因此创建多个string分段存储，便于处理。由于URL可能为绝对URL或者相对URL，所以先在在URL部分寻找”http://”，如果找到说明为绝对URL，就把这部分去掉，如果URL中还有斜杠，则剩余部分为形式为”host/path”，找到下一个斜杠位置取出host和path即可。否则host即为剩余部分，path为空。如果没有”http://”，则说明为相对URL，host部分为空，数据整个存入path中。为了补全host，接着在请求报文尝试从Host Header中获取host，先找到”Host:”位置，接着截取从”Host:”开始的剩余部分，重新读入host。检查host中是否存在端口号部分，如果存在则删除。至此，便完成了对一个HTTP请求报文的处理，得到了目标文件的主机名与路径。

我们重新使用Winsock库，利用获得的主机名，我们使用 getaddrinfo 解析目标 host 的 IP 地址，并创建一个新的套接字 originSock 来连接到目标服务器的 80 端口，与目标服务器建立连接，只需要将从目标服务器获得的数据转发给浏览器即可。这样便完成了一个普通的HTTP代理服务器。

在此基础上，我们继续为代理服务器添加 Cache 功能。为了防止文件名冲突引起的 bug，我

们根据主机名和路径生成一个唯一的缓存文件名，并检查该文件是否在 CACHE_DIR 目录中存在。如果缓存文件存在，使用编写的 getFileModTime 函数查询缓存文件的时间，如果也存在时间戳，我们便标记为缓存命中(本地)，并向准备发送给服务器的报文中的添加 if-modified-since 头行。上述两种条件不都满足时，则视为缓存不命中。发送准备好的请求报文，并检查服务器返回的我们才开始根据缓存命中状态执行不同的操作。如果先前本地缓存命中且在返回报文中找到”304 Not Modified”的消息，刚刚建立的服务器连接就不需要了，将其关闭。我们以二进制模式打开本地的缓存文件，循环读取其内容到缓冲区，并将其 send 回客户端 clientSock，完成后关闭两个套接字，以此实现了从本地缓存快速响应的功能。

相对地，如果本地缓存未命中或本地缓存命中但已经过期，我们必须从原服务器获取数据。我们首先将客户端发来的原始请求报文 req 完整地 send 到 originSock。接着，为了处理响应数据并实现缓存，我们设计了 forwardAndCache 函数。此函数进入一个循环，不断从 originSock 接收（recv）响应数据。在每次成功接收到一个数据块后，它会立即将其 send 到 clientSock 以转发给客户端，同时将这个数据块以二进制方式写入我们之前确定的缓存文件中。这个转发和写入的过程会一直持续，直到原服务器的响应数据全部传输完毕，连接关闭。通过前面的设计与实现，我们完成了前面要求的两个功能，在此基础上，我们进一步完成第三个功能。

想到一个比较简单的实现思路是，事先将要阻碍的用户、过滤或者重定向的网站写入文件中，在程序开始之后，读取存入文件中的信息，并建立用于过滤的set blockedUsers、blockedSites和用于重定向的map映射redirectMap。

在分析完HTTP报文之后，我们检查客户端的 IP 地址是否存在于blockedUsers集合中，如果存在，则说明该用户被禁止访问，代理服务器会立刻向其返回一个 "403 Forbidden" 响应并关闭连接。紧接着，对于网站过滤，检查解析到的 host 是否存在于 blockedSites 集合中。如果该网站在黑名单内，代理同样返回 403 错误，以此实现了对特定用户和特定网站的双重过滤。重定向机制的机制则是检查 host 是否在 redirectMap 中有条目。如果找到了匹配项，代理服务器不会转发请求，而是会构造一个 "302 Found" 响应，并将配置中对应的目标地址放入 "Location" 头部发送给客户端，浏览器收到后会自动跳转到新的网址，从而实现了网站重定向的功能。

如果一个请求通过了上述所有检查，代理便按照之前Cache的实现流程继续操作。最后，为了让代理服务器运行起来，我们编写了 main 函数和 startProxy 函数。main 函数负责在启动时创建缓存和配置目录并调用各个配置加载函数。随后，startProxy 函数初始化 Winsock 库，创建一个 listenSock 套接字，将其绑定到 8080 端口并开始监听。服务器的主循环位于 startProxy 中，它不断调用 accept 来阻塞等待新的客户端连接。每当一个客户端连接成功，为了能够并发处理多个请求而不阻塞主循环，我们创建了一个新的 std::thread 线程，并将 handleClient 函数以及新接受的 clientSock 作为参数传递给该线程。通过调用 detach，该线程将独立运行，而主循环则能立即返回去 accept 下一个连接请求，从而实现了多线程的并发代理服务。

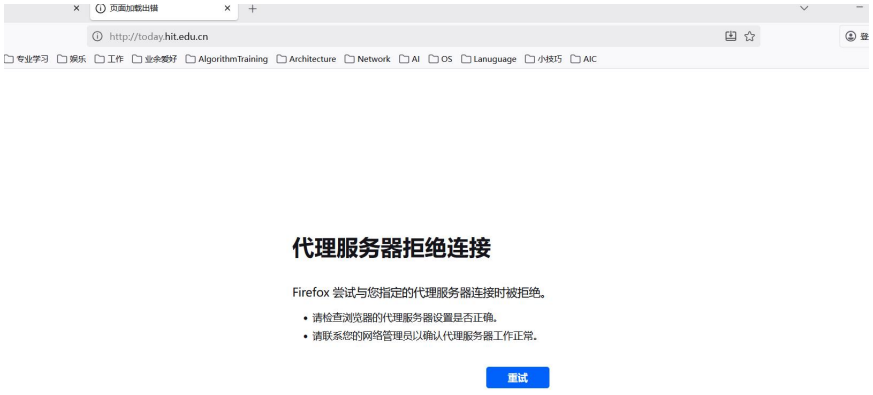
实际操作中，发现由于火狐浏览器的自动更新功能会不断发送HTTPS报文，这是之前编写的HTTP代理服务器所无法处理的，会导致代理服务器崩溃。因此在询问AI关于HTTPS的信息之后，又添加了检测和处理HTTPS请求的部分。对于HTTPS请求，选择了直接转发。由于HTTPS请求添加了CONNECT方法

实验结果：

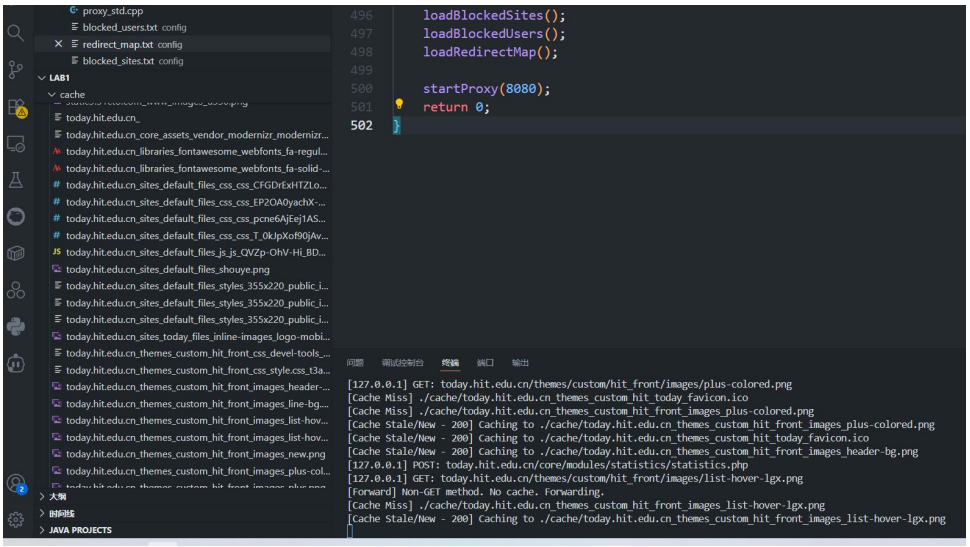
首先设置好浏览器的代理设置，本次实验中选择使用了火狐浏览器。在设置中搜索代理，会出现网络配置部分，点击设置修改HTTP代理地址为127.0.0.1，端口号为8080。



此时打开任一网页，会出现下图中的情况。



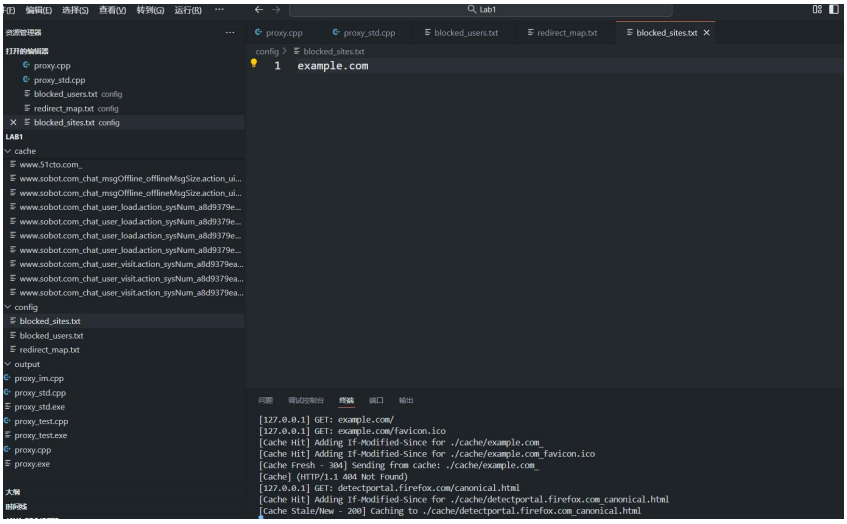
运行后重试连接，就可以看到网页又可以正常连接了。



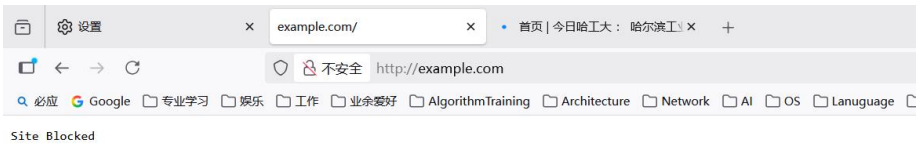
并且，由于Cache功能，在本地目录下的cache文件夹中也出现了相应的文件。

当我们再次访问http://www.today.hit.edu.cn/网站时，我们加载的速度会比第一次快很多。

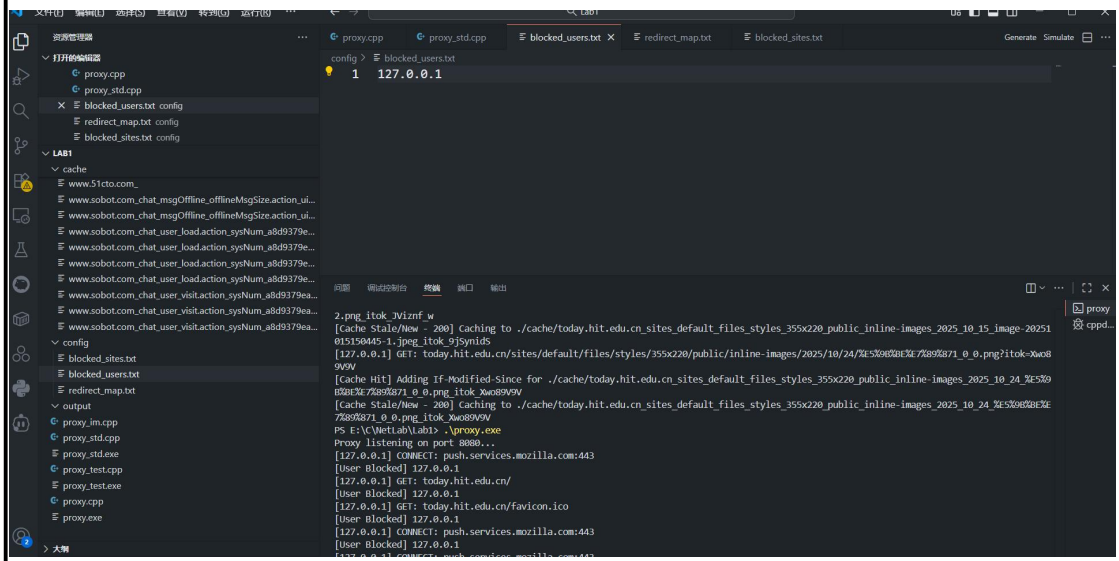
网站过滤展示：先设置好要读入的被过滤的网站，然后重新运行服务器程序。



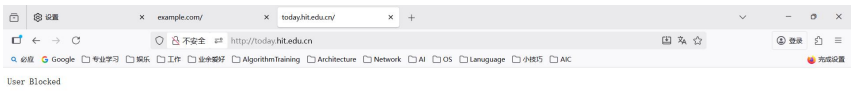
可以看到，访问example.com时，网站被过滤而无法访问，并显示了我们预设的信息。



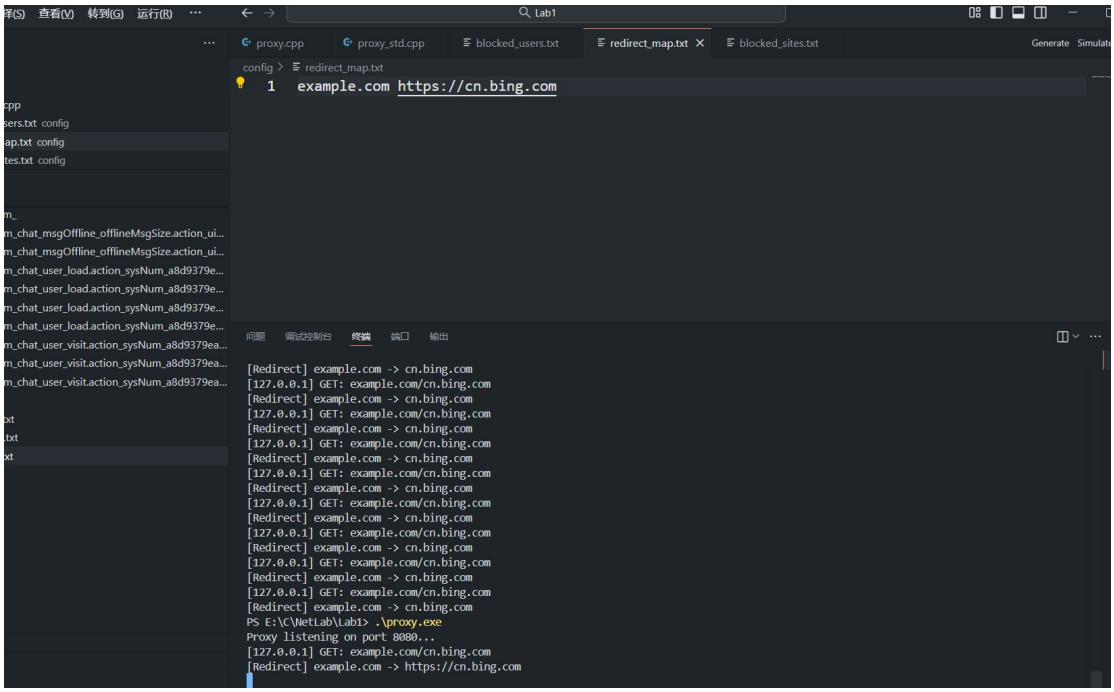
用户过滤展示：向被过滤用户名单中添加要过滤的127.0.0.1地址，启动代理服务器程序后，



就可以看到下图，当我们尝试访问今日哈工大时，界面并没有出现，而是我们预设好的信息。



重定向功能展示：向重定向表中添加服务器名与完整URL的配对，即可实现定向。



重定向效果见录制视频。

一开始设置的网站引导是访问http://example.com/重定向到 http://cn.bing.com/，然后启动程序，访问example.com，经过处理后直接转到了必应搜索的页面，至此，网站引导功能完成。(如果需要复现效果，需要注意浏览器自带的缓存，要先清除浏览器缓存才会出现效果。由于火狐浏览器的后台服务不得已添加了对HTTPS请求的处理，因此也可以访问使用HTTPS协议的网站。)

问题讨论：

在实验过程中打开网站时有时图片、数据会丢失，猜测主要原因是代理在转发响应时未按真实字节数 recv()或响应采用分块传输，未拼接完整数据。
在测试程序效果时发现程序的性能似乎不佳，在初次打开HTTP网页时会出现明显较长的加载时间，显然会影响使用体验。对于HTTPS请求，则不会出现上述情况，加载很快。可能是由于HTTP请求处理步骤较多，而HTTPS请求则选择直接转发。

心得体会：

通过本次实验，我深入理解了 HTTP 代理服务器的工作原理和缓存机制。在实现过程中，不仅掌握了网络编程中 socket 的使用方法，还复习了HTTP报文的结构，并在此基础上处理HTTP 报文。扩展任务的文件缓存机制也启发了我对程序架构设计的思考。在调试 If-Modified-Since 逻辑时，代理与服务器之间的通信并不需要额外接口，只需遵循协议头即可实现缓存协同。同时，也意识到 HTTPS 代理的复杂性与安全边界问题，这为后续进一步学习计算机网络知识打下了基础。最后的任务，借鉴了计算机科学与工程程序设计实践的实验内容——选择通过读取文件中的内容设置默认配置，这样就可以实现灵活的访问控制与重定向功能。